

Evidencias de Validación

Modelo Híbrido de Solicitudes Ciudadanas

Asignatura	Taller de Datos I Modelamiento (CD201ICDA)
Evaluación	Examen Final Transversal
Institución	Instituto Profesional San Sebastián
Fecha	09-06-2026

Este documento consolida la evidencia ejecutable que respalda la sección **Validación y Evaluación** del Informe Técnico. Reúne los resultados de las pruebas de integridad relacional (PostgreSQL), validación de JSON Schema (Draft 2020-12) y verificación de patrones embebidos en el dataset sintético.

Resumen Ejecutivo

El modelo fue sometido a **437 evaluaciones automatizadas** agrupadas en tres categorías. La tasa de detección de violaciones es del 100% en las pruebas negativas diseñadas, y la tasa de aceptación de datos válidos es del 100% en las pruebas positivas masivas.

21	400	12	100%
Pruebas SQL (20 negativas + 1 positiva)	Registros JSONB validados (100% OK)	Casos JSON inválidos (100% rechazados)	Cobertura de criterios ISO/IEC 25012

Criterios de calidad cubiertos (ISO/IEC 25012)

Criterio	Mecanismos del modelo	Pruebas que lo verifican
Precisión	CHECK constraints, regex de RUT y email, dominios enumerados, rangos numéricos	B.1.1 – B.1.11 (SQL) + INV-002, 006 (JSON)
Compleitud	NOT NULL, CHECK length, CHECK semántico (email o teléfono)	B.2.1 – B.2.4 (SQL) + INV-001 (JSON)
Consistencia	Integridad referencial FK, UNIQUE simple y compuesto, ON DELETE CASCADE	B.3.1 – B.3.5 (SQL) + INV-011 (JSON)
Claridad	COMMENT ON SQL, JSON Schema descriptivo, Diccionario de Datos formal	Documentación + meta-validación del Schema

1. Pruebas de Integridad Relacional (SQL)

Las pruebas están definidas en Pruebas_Integridad.sql y se ejecutan en PostgreSQL 15+ mediante psql. Cada caso negativo está envuelto en un bloque SAVEPOINT ... ROLLBACK TO SAVEPOINT para que la transacción no aborte y los 21 tests puedan ejecutarse en un solo run. La salida esperada del cliente es 20 mensajes *ERROR* + 1 confirmación de inserción válida.

ID	Descripción	Criterio	Restricción verificada	Resultado
A.1	Inserción válida de ciudadano + solicitud completa	Positiva	—	OK · Acepta
B.1.1	RUT con formato inválido (sin guión / DV)	Precisión	CHECK regex chileno	OK · Rechaza
B.1.2	Email con formato inválido	Precisión	CHECK regex RFC email	OK · Rechaza
B.1.3	Prioridad fuera del dominio (CRITICA)	Precisión	CHECK IN enumerado	OK · Rechaza
B.1.4	Estado fuera del dominio enumerado	Precisión	CHECK IN enumerado	OK · Rechaza
B.1.5	Tipo de archivo no permitido en adjuntos	Precisión	CHECK IN enumerado	OK · Rechaza
B.1.6	Canal fuera del dominio enumerado	Precisión	CHECK IN enumerado	OK · Rechaza
B.1.7	fecha_cierre anterior a fecha_creacion	Precisión	CHECK temporal	OK · Rechaza
B.1.8	fecha_limite no posterior a fecha_creacion	Precisión	CHECK temporal	OK · Rechaza
B.1.9	SLA negativo en tipo_solicitud	Precisión	CHECK > 0	OK · Rechaza
B.1.10	Capacidad diaria de cuadrilla = 0	Precisión	CHECK > 0	OK · Rechaza
B.1.11	Tamaño de adjunto negativo	Precisión	CHECK >= 0	OK · Rechaza
B.2.1	Solicitud sin descripción	Compleitud	NOT NULL	OK · Rechaza
B.2.2	Solicitud con descripción < 10 caracteres	Compleitud	CHECK length(descripcion) >= 10	OK · Rechaza
B.2.3	Ciudadano sin email NI teléfono	Compleitud	CHECK (email OR telefono)	OK · Rechaza
B.2.4	Solicitud sin estado_actual_id	Compleitud	NOT NULL	OK · Rechaza
B.3.1	FK a ciudadano inexistente	Consistencia	FK REFERENCES ciudadano	OK · Rechaza
B.3.2	FK a tipo_solicitud inexistente	Consistencia	FK REFERENCES tipo_solicitud	OK · Rechaza
B.3.3	Folio duplicado	Consistencia	UNIQUE (folio)	OK · Rechaza
B.3.4	RUT de ciudadano duplicado	Consistencia	UNIQUE (rut)	OK · Rechaza
B.3.5	Funcionario con unidad inexistente	Consistencia	FK REFERENCES unidad_municipal	OK · Rechaza

Resultado consolidado: 20/20 pruebas negativas producen ERROR de PostgreSQL y la prueba positiva A.1 inserta correctamente. La transacción global termina con ROLLBACK, por lo que el estado de la BD no cambia.

2. Validación de JSON Schema

El JSON Schema `JSON_Schema.json` (Draft 2020-12) se aplica al campo `solicitud.metadatos.JSONB`. El validador `validar_json.py` ejecuta tres niveles de prueba:

2.1 Meta-validación del propio Schema

Se invoca `Draft202012Validator.check_schema(schema)`. El método no lanza excepción, lo que confirma que el Schema es por sí mismo una instancia válida de la meta-schema de Draft 2020-12.

2.2 Validación masiva (pruebas positivas)

Los **400 registros sintéticos** generados por `generar_datos.py` se validan uno a uno contra el Schema. Resultado: **400/400 (100%)** aceptados. Esto confirma que el generador y el Schema están sincronizados.

2.3 Pruebas negativas diseñadas

Doce instancias inválidas, cada una vulnerando una regla distinta del Schema. El validador debe rechazarlas todas y reportar la regla específica violada.

ID	Descripción	Categoría	Mensaje del validador (Draft 2020-12)
INV-001	Falta el bloque 'geolocalizacion'	Required ausente	'geolocalizacion' is a required property
INV-002	Latitud fuera del rango de Chile (lat > -17.5)	Rango numérico	10.0 is greater than the maximum of -17.5
INV-003	Longitud como string en vez de number	Tipo de dato	'no-numero' is not of type 'number'
INV-004	IP con formato inválido	Formato (ipv4 ipv6)	'no-es-ip' is not valid under any of the given schemas
INV-005	Tipo de atributos fuera del enum (ALIENIGENA)	Enum (discriminador)	'ALIENIGENA' is not one of [BACHE, LUMINARIA, ...]
INV-006	BACHE con profundidad fuera de rango (>100 cm)	Rango numérico	500 is greater than the maximum of 100
INV-007	BACHE con calzada fuera del enum	Enum	'ADOQUIN' is not one of [ASFALTO, HORMIGON, TIERRA]
INV-008	BACHE con campo extra no permitido	additionalProperties	'color_baldosa' was unexpected
INV-009	LUMINARIA con numero_poste mal formado	Pattern (regex)	'POSTE-X' does not match '^P-[0-9]{4}\$'
INV-010	BASURA con riesgo_sanitario como string	Tipo de dato	'si' is not of type 'boolean'
INV-011	Mezcla: tipo=BACHE con campos de LUMINARIA	Discriminador	'numero_poste', 'tipo_falla' were unexpected
INV-012	version_app con formato no semver	Pattern (regex)	'v2' does not match '^([0-9]+\\.([0-9]+\\.([0-9]+)\$))'

Resultado consolidado: 12/12 instancias inválidas correctamente rechazadas. La cobertura abarca 8 categorías distintas de error: required ausente, rango numérico, tipo de dato, formato (ipv4), enum, additionalProperties, pattern (regex) y discriminador con if/then/else.

3. Validación de Patrones Inyectados

El generador de datos sintéticos no produce registros aleatorios uniformes. Inyecta cinco patrones controlados que después permiten verificar si las consultas analíticas del modelo los detectan correctamente. Esto es lo que conecta el modelamiento técnico con la **utilidad operativa y estratégica** que pide el caso.

Patrón	Esperado	Medido en el dataset	Estado
Reincidencia (top 5 ciudadanos)	~40%	39.2% (157/400)	OK · Detectable
Concentración sectorial (CENTRO)	~35%	33.5% (134/400)	OK · Detectable
Retraso de SLA en solicitudes cerradas	~30%	33.9% (107/316)	OK · Detectable
Estacionalidad de BACHE (jun-ago)	↑	Sí (sesgo aplicado)	OK · Detectable
Mapeo tipo→unidad consistente	100%	100% (400/400)	OK · Verificado

Cada patrón está enlazado con una de las cinco necesidades del cliente declaradas en el enunciado (§ Análisis de Necesidades del Informe Técnico):

Patrón	Necesidad del cliente que valida	Consulta del modelo que lo detecta
Reincidencia	Detectar perfiles de alta reincidencia	Consultas.sql · B.2
Concentración sectorial	Patrones de reclamos por sector y tipo	Consultas.sql · B.1, B.5
Retraso de SLA	Reducir tiempos de respuesta	Consultas.sql · B.3, B.4
Estacionalidad de BACHE	Patrones temporales para planificación	Consultas.sql · B.3 + filtro temporal
Mapeo tipo→unidad	Optimizar asignación de cuadrillas	Consultas.sql · A.3, B.4

4. Conclusión

El modelo cumple las cuatro dimensiones de calidad de datos definidas por la norma **ISO/IEC 25012** (precisión, completitud, consistencia, claridad), demostrado por **33 pruebas negativas distintas** que el modelo rechaza de forma determinística, más **400 instancias positivas masivas** que acepta correctamente, más la verificación de que el dataset contiene los **patrones analíticos** que el caso de uso exige detectar.

Las pruebas son **reproducibles**: los archivos `Pruebas_Integridad.sql`, `validar_json.py`, `generar_datos.py` y `Notebook_Modelo.ipynb` permiten regenerar este reporte desde cero en cualquier instalación de PostgreSQL 15+ con Python 3.10+.